

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_6c18d403-385\agent-a505fb7d2d56637a5.jsonl`
- SHA-256: `f57c82f1f72f9c5048147ca8d0e6772770aaaddd65766ad8ddb0fb5dbbf99434`
- Source modified: `2026-07-06T20:59:30+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf\_6c18d403-385`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T20:55:37.587Z)

Reviewing GitHub PR #50 of the repo at C:/Users/avidu/Projects/Telegram ? "feat(policy): add F1 pipeline primitives" (branch feat/f1-policy-engine -> main).

It is the first F1 slice of ADR 0012 (policy pipeline, just merged as #40). +280/-14, 4 files:

- src/tg_video_filter/policy.py (NEW): PolicyContext, StageResult (+ pass_/fail helpers), PolicyStage Protocol, evaluate_pipeline().
- src/tg_video_filter/db.py: NEW load_policy_config/save_policy_config (raw JSON dict over policies.config_json);
load_filter_config now uses model_validate(_policy_config_json_to_dict(...)) instead of model_validate_json;
save_filter_config now MERGES cfg.model_dump(mode='json') into the existing raw config dict (preserving unknown keys) instead of replacing the whole blob with cfg.model_dump_json().
- tests/test_db.py + tests/test_policy.py: primitives tests + a preserve-unknown-keys regression test.

A checked-out copy of the PR is at C:/Users/avidu/Projects/_wt-pr50 (read files there or via 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f1-policy-engine:<path>').

Run repros with: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script> (build schema via db._apply_schema on aiosqlite ":memory:").

BACKGROUND ? why this PR exists: ADR 0012 review found that load_filter_config/save_filter_config would ERASE future pipeline keys (topics/security_checks/deep_inspect) when a user runs /set, because save re-serialized via FilterConfig (which drops unknown keys). F1 fixes that persistence boundary. The main reviewer already verified the happy path works (pipeline keys survive a /set; flat fields update). Your job: find edge cases / bugs / regressions the happy path missed.

Ground rules:

- READ actual code, cite file:line. Run repros. Authoritative gate ALREADY RUN: ruff clean, 501 passed @ 96.63%.
- Report ONLY defects in THIS PR's diff. No praise. Concrete failure scenario per finding. Rank most-severe first.
- Severity: high (real bug/regression/data-loss), medium (edge case, latent, asymmetry), low (typing/hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code, run a repro with PYTHONPATH=C:/Users/avidu/Projects/_wt-pr50/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. REFUTED if

handled / can't occur / pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled.
Corrected severity.
FINDING (primitives_scope): Asymmetric handling of a corrupt/non-object stored config: save
heals silently, load crashes
severity low | src/tg_video_filter/db.py:1759
summary: save_filter_config wraps _policy_config_json_to_dict in try/except and degrades a non-
object/corrupt config_json to {}, but load_filter_config and load_policy_config call
_policy_config_json_to_dict with no guard, so the same corrupt row raises ValueError on read.
scenario: If policies.config_json for the default policy ever holds valid JSON that is not an
object (e.g. 'null', '[]', '"x"') or is truncated, load_filter_config(conn, user_id) at line
1759 and load_policy_config at line 1320 both raise (ValueError / JSONDecodeError) ? crashing
the manager startup path (manager.py:140) and /set read path (command_service.py) ? whereas a
subsequent save_filter_config on the same row would silently reset it to {} and rewrite flat
defaults. The two functions disagree on whether a bad row is fatal.
evidence: Repro confirmed: _policy_config_json_to_dict('null'|'[]'|'"str"'|'123') raises
ValueError and '{bad}' raise JSONDecodeError; both are caught in save_filter_config
(db.py:1775) but load_filter_config (db.py:1759) and load_policy_config (db.py:1320) call the
helper directly. In normal operation stored configs are always objects, so this is latent, not
an active bug.
rec: Either make the two paths symmetric (also tolerate a corrupt row on read, falling back to
FilterConfig() / defaults) or make save fail loudly too. As-is, document that a corrupt row is
fatal-on-read but self-healed-on-write, which is surprising.

2. assistant (2026-07-06T20:55:42.334Z)

I'll verify this finding by reading the cited code and running repros.

3. user (2026-07-06T20:55:46.018Z)

done

4. assistant (2026-07-06T20:55:47.937Z)

Let me read the actual cited lines and the helper function.

5. user (2026-07-06T20:55:51.657Z)

```
1292: def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1310: async def load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str,
object]:
1320:     return _policy_config_json_to_dict(policy.config_json)
1323: async def save_policy_config(
1749: async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1759:     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1762: async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1774:         raw_config = _policy_config_json_to_dict(existing.config_json)
```

6. user (2026-07-06T20:55:54.542Z)

```
1292 def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
1293     """Decode a policy config JSON object.
1294
1295     ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
1296     expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
1297     nested stage config can coexist.
1298     """
1299     decoded = json.loads(raw)
1300     if not isinstance(decoded, dict):
1301         raise ValueError("policy config_json must decode to a JSON object")
1302     return decoded
1303
1304
1305 def _policy_config_dict_to_json(config: dict[str, object]) -> str:
```

```

1306         """Encode policy config deterministically and compactly."""
1307         return json.dumps(config, separators=(",", ":"), sort_keys=True)
1308
1309
1310     async def load_policy_config(conn: aiomysqlite.Connection, account_id: int) -> dict[str,
object]:
1311         """Load the raw default policy config as a JSON object.
1312
1313         Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
1314         as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
1315         through the flat ``FilterConfig`` model.
1316         """
1317         policy = await get_default_policy(conn, account_id)
1318         if policy is None:
1319             return FilterConfig().model_dump(mode="json")
1320         return _policy_config_json_to_dict(policy.config_json)
1321
1322
1323     async def save_policy_config(
1324         conn: aiomysqlite.Connection, account_id: int, config: dict[str, object]
1325     ) -> None:
1326         """Persist the raw default policy config object."""
1327         existing = await get_default_policy(conn, account_id)
1328         await upsert_policy(
1329             conn,
1330             Policy(
1331                 id=existing.id if existing is not None else None,
1332                 account_id=account_id,
1333                 name="default",
1334                 config_json=_policy_config_dict_to_json(config),
1335                 is_default=True,
1336                 created_at=(existing.created_at if existing is not None else _now()),
1337                 updated_at=_now(),
1338             ),
1339         )
1340
1341
1342     # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1343
1344
1345     async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1346         """Insert or update a destination row by (account, provider, target)."""
1347         await conn.execute(
1348             """INSERT INTO destinations
1349             (account_id, provider, target, kind, title, is_output, created_at)
1350             VALUES (?, ?, ?, ?, ?, ?, ?)
1351             ON CONFLICT(account_id, provider, target) DO UPDATE SET
1352             kind=excluded.kind,
1353             title=excluded.title,
1354             is_output=excluded.is_output""",
1355             (
1356                 dest.account_id,
1357                 dest.provider,
1358                 dest.target,
1359                 dest.kind,
1360                 dest.title,
1361                 int(dest.is_output),

```

7. assistant (2026-07-06T20:55:56.539Z)

Now let me read the load/save_filter_config section.

8. user (2026-07-06T20:55:59.636Z)

```

1749 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1750     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1751
1752     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1753     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1754     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1755     """
1756     policy = await get_default_policy(conn, user_id)
1757     if policy is None:
1758         return FilterConfig()
1759     return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))
1760
1761
1762 async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1763     """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
1764
1765     ADR 0012: future pipeline keys live in the same ``policies.config_json``
1766     object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
1767     merge the flat config fields into the existing object so unknown policy
1768     keys survive legacy ``/set`` command updates.
1769     """
1770     existing = await get_default_policy(conn, user_id)
1771     raw_config: dict[str, object] = {}
1772     if existing is not None:
1773         try:
1774             raw_config = _policy_config_json_to_dict(existing.config_json)
1775         except (json.JSONDecodeError, ValueError):
1776             raw_config = {}
1777     raw_config.update(cfg.model_dump(mode="json"))
1778     await save_policy_config(conn, user_id, raw_config)
1779
1780
1781 # --- bulk helpers (used by the manager on startup) -----
1782
1783
1784 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1785     conn: aiomysqlite.Connection, user_id: int, limit: int = 100
1786 ) -> list[VideoRecord]:
1787     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1788     conn.row_factory = aiomysqlite.Row
1789     async with conn.execute(
1790         """SELECT ci.*, s.provider_source_id AS src_pid
1791            FROM content_items ci
1792            JOIN sources s ON s.id = ci.source_id
1793            WHERE ci.account_id = ?
1794            ORDER BY ci.id DESC LIMIT ?""",
1795         (user_id, limit),
1796     ) as cur:
1797         rows = await cur.fetchall()
1798     return [

```

9. assistant (2026-07-06T20:56:03.752Z)

The finding is factually accurate about the code structure. The asymmetry is real:

- `save\_filter\_config` (line 1773-1776): try/except around `\_policy\_config\_json\_to\_dict`, degrading to `{}`
- `load\_filter\_config` (line 1759): no guard
- `load\_policy\_config` (line 1320): no guard

Now let me verify the repro claims and check whether this is actually within the PR diff (i.e., was `\_policy\_config\_json\_to\_dict` newly introduced, and did `load\_filter\_config` previously guard).

10. user (2026-07-06T20:56:06.550Z)

```

diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index a059e57..7f7e400 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -11,6 +11,7 @@ in-memory database (`aiosqlite.connect(":memory:")`).

from __future__ import annotations

+import json
+from collections.abc import Awaitable, Callable, Iterable
+from datetime import UTC, datetime

@@ -1288,6 +1289,56 @@ async def get_default_policy(conn: aiosqlite.Connection, account_id: int)
-> Pol
    return _row_to_policy(row) if row else None

+def _policy_config_json_to_dict(raw: str) -> dict[str, object]:
+    """Decode a policy config JSON object.
+
+    ADR 0012 keeps future policy keys in ``policies.config_json``. The value is
+    expected to be a JSON object so legacy flat ``FilterConfig`` keys and future
+    nested stage config can coexist.
+
+    """
+    decoded = json.loads(raw)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return decoded
+
+def _policy_config_dict_to_json(config: dict[str, object]) -> str:
+    """Encode policy config deterministically and compactly."""
+    return json.dumps(config, separators=(",", ":"), sort_keys=True)
+
+async def load_policy_config(conn: aiosqlite.Connection, account_id: int) -> dict[str, object]:
+    """Load the raw default policy config as a JSON object.
+
+    Unlike :func:`load_filter_config`, this preserves future ADR 0012 keys such
+    as ``topics`` / ``security_checks`` / ``deep_inspect`` instead of validating
+    through the flat ``FilterConfig`` model.
+
+    """
+    policy = await get_default_policy(conn, account_id)
+    if policy is None:
+        return FilterConfig().model_dump(mode="json")
+    return _policy_config_json_to_dict(policy.config_json)
+
+async def save_policy_config(
+    conn: aiosqlite.Connection, account_id: int, config: dict[str, object]
+) -> None:
+    """Persist the raw default policy config object."""
+    existing = await get_default_policy(conn, account_id)
+    await upsert_policy(
+        conn,
+        Policy(
+            id=existing.id if existing is not None else None,
+            account_id=account_id,
+            name="default",
+            config_json=_policy_config_dict_to_json(config),
+            is_default=True,
+            created_at=(existing.created_at if existing is not None else _now()),
+            updated_at=_now(),
+        ),
+    )

```

```

+
+
# --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) ---

@@ -1705,24 +1756,26 @@ async def load_filter_config(conn: aioredis.Connection, user_id: int)
-> Filter
    policy = await get_default_policy(conn, user_id)
    if policy is None:
        return FilterConfig()
-    return FilterConfig.model_validate_json(policy.config_json)
+    return FilterConfig.model_validate(_policy_config_json_to_dict(policy.config_json))

    async def save_filter_config(conn: aioredis.Connection, user_id: int, cfg: FilterConfig) ->
    None:
-        """Deprecated shim: upsert the account's default :class:`Policy`."""
+        """Deprecated shim: update flat ``FilterConfig`` keys on the default policy.
+
+        ADR 0012: future pipeline keys live in the same ``policies.config_json``
+        object. Do not overwrite the whole blob with ``cfg.model_dump_json()``;
+        merge the flat config fields into the existing object so unknown policy
+        keys survive legacy ``/set`` command updates.
+
+        """
        existing = await get_default_policy(conn, user_id)
-        await upsert_policy(
-            conn,
-            Policy(
-                id=existing.id if existing is not None else None,
-                account_id=user_id,
-                name="default",
-                config_json=cfg.model_dump_json(),
-                is_default=True,
-                created_at=(existing.created_at if existing is not None else _now()),
-                updated_at=_now(),
-            ),
-        )
+        raw_config: dict[str, object] = {}
+        if existing is not None:
+            try:
+                raw_config = _policy_config_json_to_dict(existing.config_json)
+            except (json.JSONDecodeError, ValueError):
+                raw_config = {}
+        raw_config.update(cfg.model_dump(mode="json"))
+        await save_policy_config(conn, user_id, raw_config)

# --- bulk helpers (used by the manager on startup) -----
@@ -1789,6 +1842,8 @@ __all__ = [
    "get_content_item_by_dedup_key",
    "upsert_policy",
    "get_default_policy",
+    "load_policy_config",
+    "save_policy_config",
    # Destination / Routing / Delivery repos (ADR 0011, Track C)
    "upsert_destination",
    "get_destination",

```

11. assistant (2026-07-06T20:56:09.406Z)

Now I understand the pre-existing behavior. Before this PR, `load\_filter\_config` used `FilterConfig.model\_validate\_json(policy.config\_json)`. Let me check what that did on a non-object/corrupt config, to assess whether the "crash on load" is a NEW regression or pre-existing.

12. assistant (2026-07-06T20:56:17.687Z)

```
import asyncio, json
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import FilterConfig, Policy
```

Behavior of pre-PR path: model_validate_json on various corrupt values

```
print("=== pre-PR: FilterConfig.model_validate_json ===")
for v in ['null', '[]', '"x"', '123', '{bad', '']:
    try:
        FilterConfig.model_validate_json(v)
        print(f"{v!r}: OK")
    except Exception as e:
        print(f"{v!r}: {type(e).__name__}: {str(e)[:60]}")

print("=== PR helper: _policy_config_json_to_dict ===")
for v in ['null', '[]', '"x"', '123', '{bad', '']:
    try:
        db._policy_config_json_to_dict(v)
        print(f"{v!r}: OK")
    except Exception as e:
        print(f"{v!r}: {type(e).__name__}: {str(e)[:60]}")

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    # insert a default policy with a non-object config_json
    now = db._now()
    await db.upsert_policy(conn, Policy(id=None, account_id=1, name="default",
        config_json="null", is_default=True, created_at=now, updated_at=now))
    print("=== load_filter_config on config_json='null' ===")
    try:
        c = await db.load_filter_config(conn, 1)
        print("OK", c)
    except Exception as e:
        print(f"{type(e).__name__}: {str(e)[:80]}")
    print("=== load_policy_config on config_json='null' ===")
    try:
        c = await db.load_policy_config(conn, 1)
        print("OK", c)
    except Exception as e:
        print(f"{type(e).__name__}: {str(e)[:80]}")

asyncio.run(main())
```

13. user (2026-07-06T20:56:17.687Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

14. user (2026-07-06T20:58:27.755Z)

Command running in background with ID: b348i38g5. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\b348i38g5.output. You will be notified when it completes. To check interim output, use Read on that file path.

15. user (2026-07-06T20:58:35.382Z)

```
=== pre-PR: FilterConfig.model_validate_json ===
'null': ValidationError: 1 validation error for FilterConfig
  Input should be an obj
'[]': ValidationError: 1 validation error for FilterConfig
  Input should be an obj
```

```

"x": ValidationError: 1 validation error for FilterConfig
  Input should be an obj
'123': ValidationError: 1 validation error for FilterConfig
  Input should be an obj
'{bad}': ValidationError: 1 validation error for FilterConfig
  Invalid JSON: key must
': ValidationError: 1 validation error for FilterConfig
  Invalid JSON: EOF while
=== PR helper: _policy_config_json_to_dict ===
'null': ValueError: policy config_json must decode to a JSON object
'[]': ValueError: policy config_json must decode to a JSON object
"x": ValueError: policy config_json must decode to a JSON object
'123': ValueError: policy config_json must decode to a JSON object
'{bad}': JSONDecodeError: Expecting property name enclosed in double quotes: line 1 co
': JSONDecodeError: Expecting value: line 1 column 1 (char 0)
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 41, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 26, in main
    await db.upsert_policy(conn, Policy(id=None, account_id=1, name="default",
        config_json="null", is_default=True, created_at=now, updated_at=now))
  File "C:\Users\avidu\Projects\_wt-pr50\src\tg_video_filter\db.py", line 1253, in upsert_policy
    await conn.execute(
    ...<15 lines>...
    )
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 223,
in execute
    cursor = await self._execute(self._conn.execute, sql, parameters)
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 160,
in _execute
    return await future
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 63, in
_connection_worker_thread
    result = function()
sqlite3.IntegrityError: FOREIGN KEY constraint failed

```

16. assistant (2026-07-06T20:58:39.220Z)

The helper repro confirms the finding's evidence exactly. But my upsert failed on a FK constraint (needs an account row). More importantly, the pre-PR path (`model\_validate\_json`) ALSO raised on every corrupt value. So the "crash on load" is NOT a new regression ? it's pre-existing behavior. Let me fix the FK and confirm the load path, then assess the regression angle.

17. user (2026-07-06T20:58:42.065Z)

```

200:async def _apply_schema(conn: aiosqlite.Connection) -> None:
283:     CREATE TABLE accounts (

```


Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 1044, in
_bootstrap_inner
    self.run()
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 995, in
run
    self._target(*self._args, **self._kwargs)
    ~~~~~^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 75, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_exception, future, e)
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

20. assistant (2026-07-06T20:59:12.148Z)

The repro fully confirms the described behavior (the trailing exception is just aiosqlite teardown noise, irrelevant). The asymmetry ships exactly as described:

- ``load_filter_config`` and ``load_policy_config`` raise on corrupt/non-object `config_json`.
- ``save_filter_config`` swallows the same corruption to ``{}`` and self-heals.

Now the key adversarial question for severity: ****is the "crash on load" a new regression introduced by this PR, or pre-existing?****

Pre-PR, ``load_filter_config`` used ``FilterConfig.model_validate_json(policy.config_json)``, which ? as my first repro showed ? ALSO raised (`ValidationError/Invalid JSON`) on all the same corrupt values (``null``, ``[]``, ``"x"``, ``123``, ``{bad``, `` ```). So `load-crashes-on-corrupt-row` is pre-existing behavior; this PR did not make the read path newly fatal. What the PR newly introduced is:

1. The ****asymmetry**** itself: ``save_filter_config`` newly gained a try/except that heals corruption to ``{}``, while the read paths stayed fatal. Pre-PR save also re-serialized the whole blob via ``cfg.model_dump_json()`` and never read the existing config, so there was no read-decode in save to guard. So the asymmetry (heal-on-write vs fatal-on-read) is genuinely new in this diff.
2. ``load_policy_config`` is a brand-new function that is fatal-on-corrupt-row.

So the defect that genuinely ships in THIS diff is the asymmetry, not a data-loss regression. The load-crash existed before. The finding itself scopes this correctly ("latent, not an active bug") and requests either symmetry or documentation. This is a real, in-diff inconsistency but low impact ? in normal operation `config_json` is always an object, and the pre-PR code was equally fatal on read.

Let me confirm one thing the finding claims: that these load paths are actually reached at startup/``/set`` (`manager.py:140`, `command_service`). Quick check that they're wired in ? though even if wired, the trigger requires an already-corrupt row which normal writes never produce.

21. user (2026-07-06T20:59:13.852Z)

```
C:/Users/avidu/Projects/_wt-pr50/src/tg_video_filter/manager.py:140:          cfg = await
db.load_filter_config(conn, session.account_id)
```

22. assistant (2026-07-06T20:59:18.284Z)

``load_filter_config`` is indeed called at manager startup (`manager.py:140`), so a corrupt row would surface there. But that crash path is pre-existing (pre-PR ``model_validate_json`` raised

identically). The genuinely-new element in this diff is the asymmetry.

Verdict: the finding is factually accurate and the asymmetry genuinely ships in this diff, but it is latent (requires a corrupt row that normal writes never produce) and the fatal-on-read behavior is pre-existing, not a regression. Severity low is correct.

23. user (2026-07-06T20:59:29.997Z)

Structured output provided successfully